Assignment 1 Git & GitHub Fundamentals

Session 1 Group D Technical Report

Full Name: Nizar Essaoudi
Student ID: 150306
Course Code & Section: CSC332403 — Group D
GitHub Username: essaoudinizar

GitHub Repository URL:
https://github.com/essaoudinizar/git-session1-essaoudinizar

1. Repository Setup

I started the assignment using the provided starter files. I opened Command Prompt in the project folder and initialized a Git repository using:

git init -b main

I created a .gitignore file to exclude environment configuration and regular log files while keeping important.log available for tracking.

The .gitignore contained:

local.env

*.log

!important.log

I verified the configuration using git status --short --untracked-files=all. Git displayed the project files, including important.log, without listing local.env, app.log, or debug.log.

This confirmed that my ignore rules were working correctly.

2. README & Commit History

I created a README.md file containing my name, GitHub username, project description, technologies used, a Git command example, a link to the official Git documentation, and the provided project banner image.

I organized my modifications into separate, meaningful commits.

Commit history:

Commit ID	Description
f121bc4	Initial commit: add starter project files

12634da Add project README documentation

3c72eb3 Update project overview section

d2481ae Improve project overview styling

4d75d72 Add About section to website

c32334a Update README.md

901ceaf Document Git diff learning

I used git log --oneline --graph --all --decorate to inspect the history.

I completed seven commits, exceeding the required minimum of five.

3. Selective Staging

To demonstrate selective staging, I modified two tracked files: index.html and style.css.

First, I added a paragraph describing Git version control and selective staging to index.html. I also updated style.css to add a blue left border to the project overview section.

I staged only the HTML file using:

git add index.html

The git status command showed:

Changes to be committed:

modified: index.html

Changes not staged for commit:

modified: style.css

I then committed the HTML change using:

git commit -m "Update project overview section"

After committing, style.css remained modified but uncommitted. I later staged and committed the CSS file separately.

This exercise helped me understand how the staging area allows developers to select specific changes for a commit.

4. Branch & Merge

I created a separate branch for the About feature using:

git switch -c feature-about

On this branch, I modified index.html by adding an About This Project section explaining the purpose of the website.

I committed the feature using:

git commit -m "Add About section to website"

After completing the feature, I returned to the main branch:

git switch main

I merged the feature branch using:

git merge feature-about

Git completed the operation using a fast-forward merge.

I verified the result using:

git log --oneline --graph --all --decorate

This demonstrated how branches support the development of features separately before integrating them into the main project.

5. Remote Changes

I created a public GitHub repository named git-session1-essaoudinizar.

I connected my local repository to GitHub using:

git remote add origin https://github.com/essaoudinizar/git-session1-essaoudinizar.git

I published the main branch using:

git push -u origin main

Difference between commit and push:

A commit saves staged changes in the local Git repository, while a push uploads local commits to a remote repository such as GitHub.

To practice synchronization, I edited README.md directly on GitHub and committed the change there.

I returned to my local repository and executed:

git fetch origin

This downloaded information about the remote changes without modifying my local working files.

Running git status showed:

Your branch is behind 'origin/main' by 1 commit,

and can be fast-forwarded.

I then executed:

git pull origin main

This integrated the remote commit into my local main branch.

Difference between fetch and pull:

Fetch updates local information about the remote repository without integrating changes into the current branch. Pull fetches and integrates remote changes.

Finally, I used git diff notes.txt to inspect an unstaged modification and git diff --staged to inspect the same modification after staging it.

I committed the notes update and pushed it to GitHub successfully.

6. Fair Use of AI

I used ChatGPT as a learning assistant while completing this assignment. I asked questions about Git commands, repository initialization, .gitignore rules, staging, commits, branches, merging, and GitHub synchronization.

ChatGPT helped me understand the purpose of commands and suggested examples for modifying the starter project. I reviewed the suggestions and executed the commands myself in Windows Command Prompt.

I checked the results using Git commands such as git status, git diff, git log, and git remote -v. When commands produced errors, I reviewed the explanations and corrected my commands before continuing.

I used AI for guidance and troubleshooting while verifying the actual results of my work.

7. Sources & Reflection

Sources used:

- Git & GitHub Session 1 lecture slides.
- Assignment 1 instructions and report template.
- Git documentation: https://git-scm.com/docs
- GitHub: https://github.com
- ChatGPT for explanations and troubleshooting.

Reflection:

The hardest concept for me was selective staging because I initially needed to understand the difference between modifying, staging, and committing a file. The exercise with index.html and style.css helped me understand how Git manages changes independently.

If Git behaves unexpectedly, I would first use git status to check the current branch and the state of my files. I would also inspect the command output and use git log or git diff to investigate the problem.

GitHub Repository:
https://github.com/essaoudinizar/git-session1-essaoudinizar